Welcome!

- 学无止境 (Learning is limitless)
- 学如逆水行舟，不进则退 (Studying is like sailing against the current: a boat must forge ahead or it will be swept downstream)
- 一分耕耘, 一分收获 (No pain, no gain)

University of Glasgow

Lecture 1: Introduction to Embedded Processors

UESTC 2004: Embedded Processors

Dr Mark D. Butala

Senior Lecturer

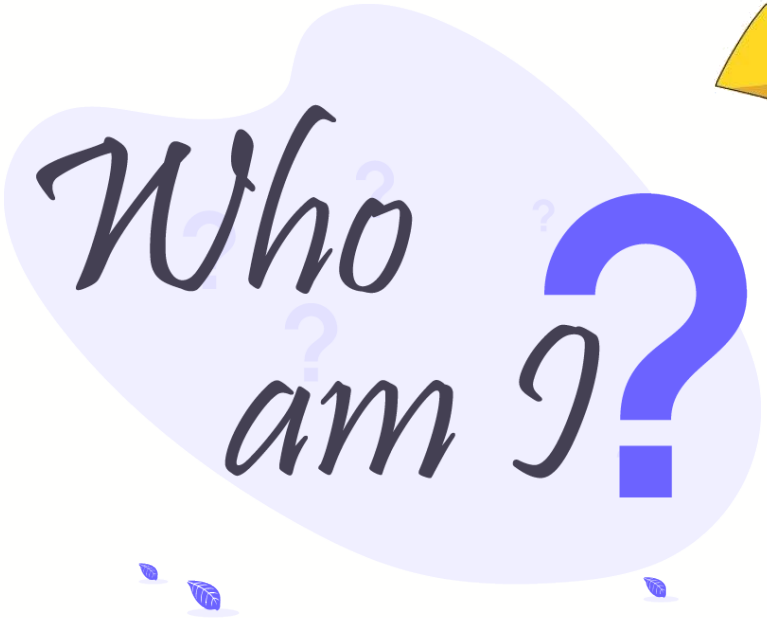
Most of the slides were prepared by Dr Ahmad Taha, Dr Yao Sun, and Dr Lina Mohjazi

WORLD CHANGING GLASGOW

A WORLD TOP 100 UNIVERSITY

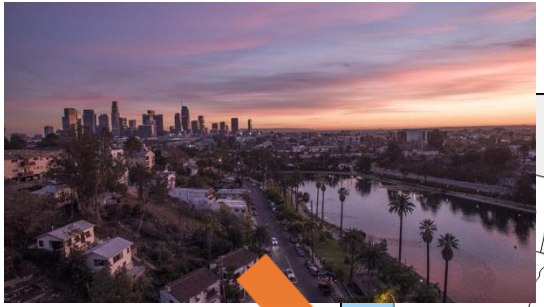
Who am I?

- Dr. Mark D. Butala, Senior Lecturer, Glasgow College UESTC.
- Email: mark.butala@glasgow.ac.uk.
- Email me to make an appointment.
- University Profile: https://www.gla.ac.uk/schools/engineering/staff/markbutala/



A Little About Me

Born in Minnesota



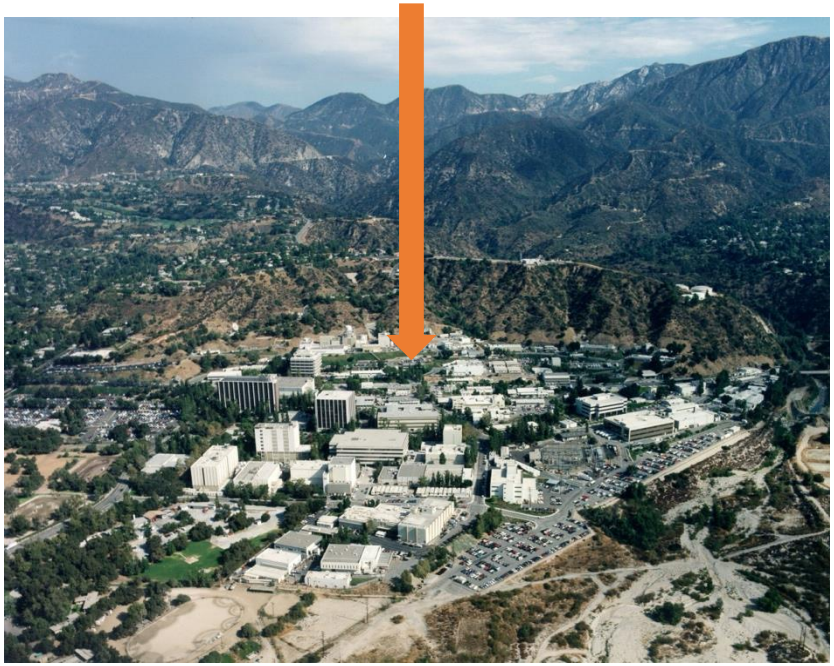
First post-Ph.D. job near Los Angeles

High school and Undergraduate degree in Delaware

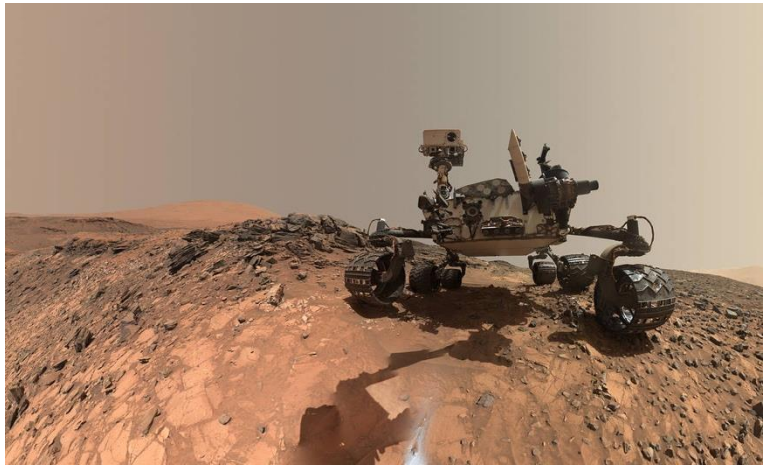
Masters and Ph.D. in Illinois

My First Job After Completing the Ph.D.

- Staff Researcher: 2010 — 2016



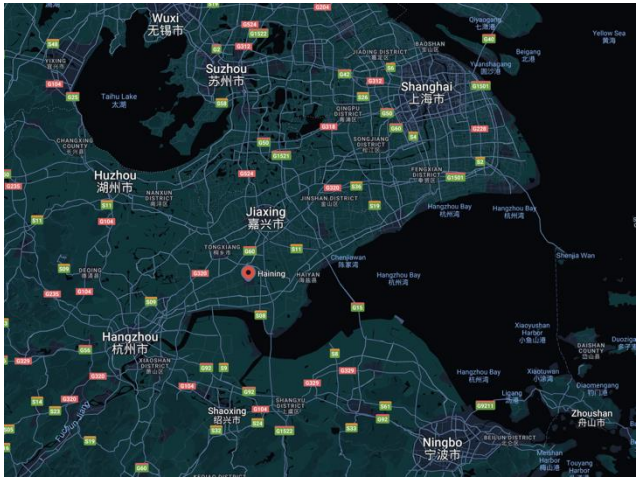
Jet Propulsion Laboratory
California Institute of Technology



Successful landing: August 6, 2012

Zhejiang University / University of Illinois Urbana-Champaign Joint Institute

- Location: Haining City (Jiaxing)
- Assistant Professor: 2017 — 2023
- Associate Professor: 2024



For Questions during the lecture

For Q&A – please **raise your hand and ask**

Lesson Plan

Aim

- Introduce UESTC2004
- Provide an overview of Embedded Systems

Learning Objectives

By the end of this lesson you will be able to:

- Define an Embedded System
- Recall the key components of an Embedded System

Assumed Previous Knowledge

- Good recall of UESTC1008 (MES)

Click for Course Specifications & Learning Outcomes

Teaching Team

The teaching team are here to help and can be contacted via email.

Dr. Mark Butala
(mark.butala@glasgow.ac.uk)

Dr. Yawar Abbas
(Yawar.Abbas@glasgow.ac.uk)

Dr. Sajjad Hussain
(Sajjad.Hussain.2@glasgow.ac.uk)

Dr. Luiz Felipe Aguinsky
(Luiz.Aguinsky@glasgow.ac.uk)
(Acting Course Coordinator for Hainan)

Course Aims

- Describe the computer architecture of microcontrollers;
- Introduce embedded systems and applications;
- Develop an understanding of the operating principles of microcontrollers and the functional interactions between software and hardware peripherals in embedded systems;
- To expose students to fundamental problems and operating principles of typical embedded systems;
- To develop experience programming embedded systems.

Intended Learning Outcomes

By the end of this course students will be able to:

- Describe the software and hardware building blocks of embedded system architecture and the various types of peripherals used in embedded systems.
- Understand/Explain the principles of embedded systems' development, from specification through to validation.
- Apply the gained knowledge to write and execute code for embedded systems, e.g., Interrupt Service Routines (ISRs).
- Evaluate trade-offs and choices in embedded system design, considering factors like power consumption.
- Synthesise the knowledge to design, implement, test, and verify embedded systems, demonstrating problem-solving abilities.

Method of Delivery

- In this course there will be 12 lectures delivered throughout the semester (New Weekly Model). All lectures will be delivered face-to-face. Each lecture will last for 1 hour and 30 mins.

- There will be four laboratory sessions timetabled throughout the semester during which four experiments, in total, have to be completed. Each session will last 1 hour and 30 minutes.

- You must attend all lectures and laboratory sessions and attendance will be recorded.

Course Teaching Schedule

Week		Topic	UoG Teaching Staff
1	Mon	Lecture 1: Introduction to Embedded Processors and Refresher	Dr. Mark Butala
	Tue		
2	Mon	Lecture 2: ARM Processors and Intro to Interfacing	
	Tue		
3	Mon	Lecture 3: Embedded System Implementation: Analogue-to-Digital Converter (ADC) & Interrupts	
	Tue		
4	Mon	Lecture 4: Embedded System Implementation: Digital-to-Analogue Converter (DAC) & Direct Memory Access (DMA)	
	Tue		
		Lab 1*: Embedded C and Software/Hardware Debugging Techniques	
5		No Lectures and Labs	
6	Tue	Lecture 5: Assembly and Machine Language	Dr. Yawar Abbas
7	Mon	Lecture 6: Project Development Cycle	
	Tue		
8	Mon	Lecture 7: Testing and debugging Embedded Systems Part 1	Dr. Sajjad Hussain
	Tue		
		Lab 2*: Application of Assembly Language and Testing/Debugging an Embedded System	
9	Mon	Lecture 8: Testing and Debugging Embedded Systems Part 2	
	Tue		

Course Teaching Schedule

10		No Lectures or Labs	
11	Mon	Lecture 9: Embedded System Implementation: Serial Peripheral Interface (SPI) Sensor Interfacing	Dr. Luiz Felipe Aguinsky
	Tue		
		Lab 3*: Peripherals Programming on mbed Part 1: Interfacing ADXL345 accelerometer module	
12	Mon	Lecture 10: Embedded System Implementation: Two-wire Interface (I^2C) Sensor Interfacing	
	Tue		
13	Mon	Lecture 11: Modular Programming	
	Tue		
		Lab 4*: Peripherals Programming on mbed Part 2: Interfacing TMP102 digital temperature module	
14	Mon	Lecture 12: Energy Efficient Embedded Systems	
	Tue		

How will you be Assessed?

Final Examination

- **75%** of the total mark

Lab work

- **15%** of the total mark

Written assessment

- **10%** of the total mark

How will you be Assessed?

- There will be 3 assessments:
  1. Written Assignment. This will involve completing a mini project in the form of designing and implementing a small embedded system. Students will submit a demo video showing the working system and a written technical report. The whole continuous assessment is worth 10% of the total mark. (announced >= Week #8)
  2. Lab work. Lab assessment will be carried out based on the successful demonstration and reflection on the lab tasks completed during the lab allocated time. The labs will help the students work towards a mini project which is done for the Written Assignment as explained in point 1 above. The whole lab work is worth 15% of the total mark.
     - Labs 1 & 2 are to be submitted, through Moodle, at the end of Week #5 and #9, respectively.
     - Labs 3 and 4 are to be submitted, through Moodle, at the end of Week #12 and #14, respectively.
  - Final Examination. There will be an end of course final examination worth 75% of the total mark.

Learning Strategy and Passing the Course

The main learning strategy to pass the course are:

1. Review the material on Moodle every week before the lecture and attempt all example sheets when provided.
2. Your lecturers want to help you. If you are finding something difficult do not hesitate to contact the relevant teaching staff.
3. Attend lectures regularly. Do not get left behind. The semester will go quickly, and you will not catch up.
4. Attendance at Labs is compulsory. Your credit will be withheld if you do not attend and attempt all labs sessions and get your lab work signed by a GTA.
5. Spend time doing background reading and attempting problems in textbooks. Ask for help if you need it!
6. Make sure you can explain in writing what you are doing. If the marker cannot understand what you are doing you will not pass.
7. If you are not sure what is required ask your lecturer. Keep asking until you are certain.

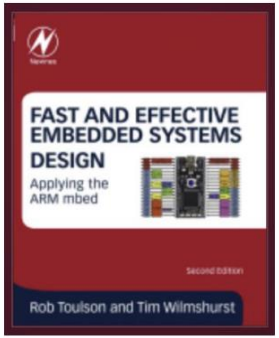
Key References

Computer organization and design: the hardware/software interface

Patterson, David A; Hennessy, John L; Alexander, Perry

The Morgan Kaufmann series in computer architecture

and design, 2014, Fifth edition.

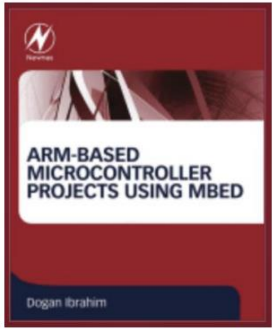


Fast and effective embedded systems design:
applying the ARM mbed

Toulson, Rob; Wilmshurst, Tim

Elsevier, 2017, Second edition.

ISBN 978-0-08-100880-5



ARM-Based microcontroller projects using MBED

Dogan Ibrahim

Elsevier, 2019.

ISBN 978-0-08-102969-5

Other Key Contacts

Teaching office contact: hainan-admin@glasgow.ac.uk

- **Chenyang Wang** Chenyang.Wang@glasgow.ac.uk

Lead GTA contact:

- **Shuangyang Xie** Shuangyang.Xie@glasgow.ac.uk

Moodle Page

Piazza

We will use Piazza for Q&A outside of lecture

- You can post completely anonymously
- The instructors and GTAs will actively address questions and concerns
- You can answer other student questions!
- Piazza is a free service, NEVER “contribute” RMB to the company

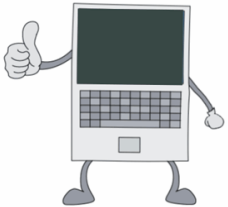
Piazza registration (**use your UofG email ID**):

Questions?

Got a question, comment, concern? 🙋‍♀️

Ask me anything! (embedded processors related 😎**)**

Different Terms "???? Computing"

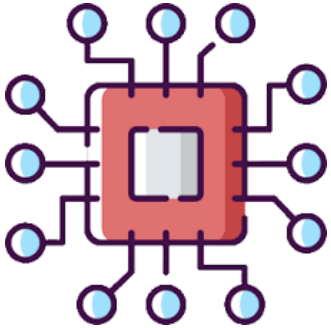


- Mainframe computers for information processing (1980s).
- Ubiquitous Computing: A term created/coined by Mark Weiser. *Meaning?* computing (and information) **anytime, anywhere** (1988).
- Invisible Computing: computers integrated into products (1988).
- Other terms: pervasive computing & ambient intelligence (*capturing and processing data through sensors, processors, and actuators*).

Different Terms "???? Computing"

- Miniaturisation enabled the integration of information processing and the physical environment using computers.

- This led to the development of **embedded systems**.



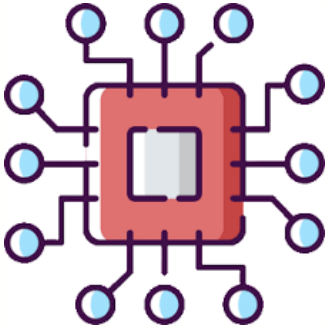
- Definition (Peter Marwedel, 2003):
  - *"Embedded systems are information processing systems embedded into enclosing products."*

Embedded Systems

Mentimeter

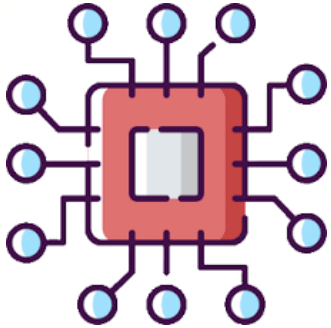
What is an Embedded System?

Slide Will be Revealed
During the Lecture

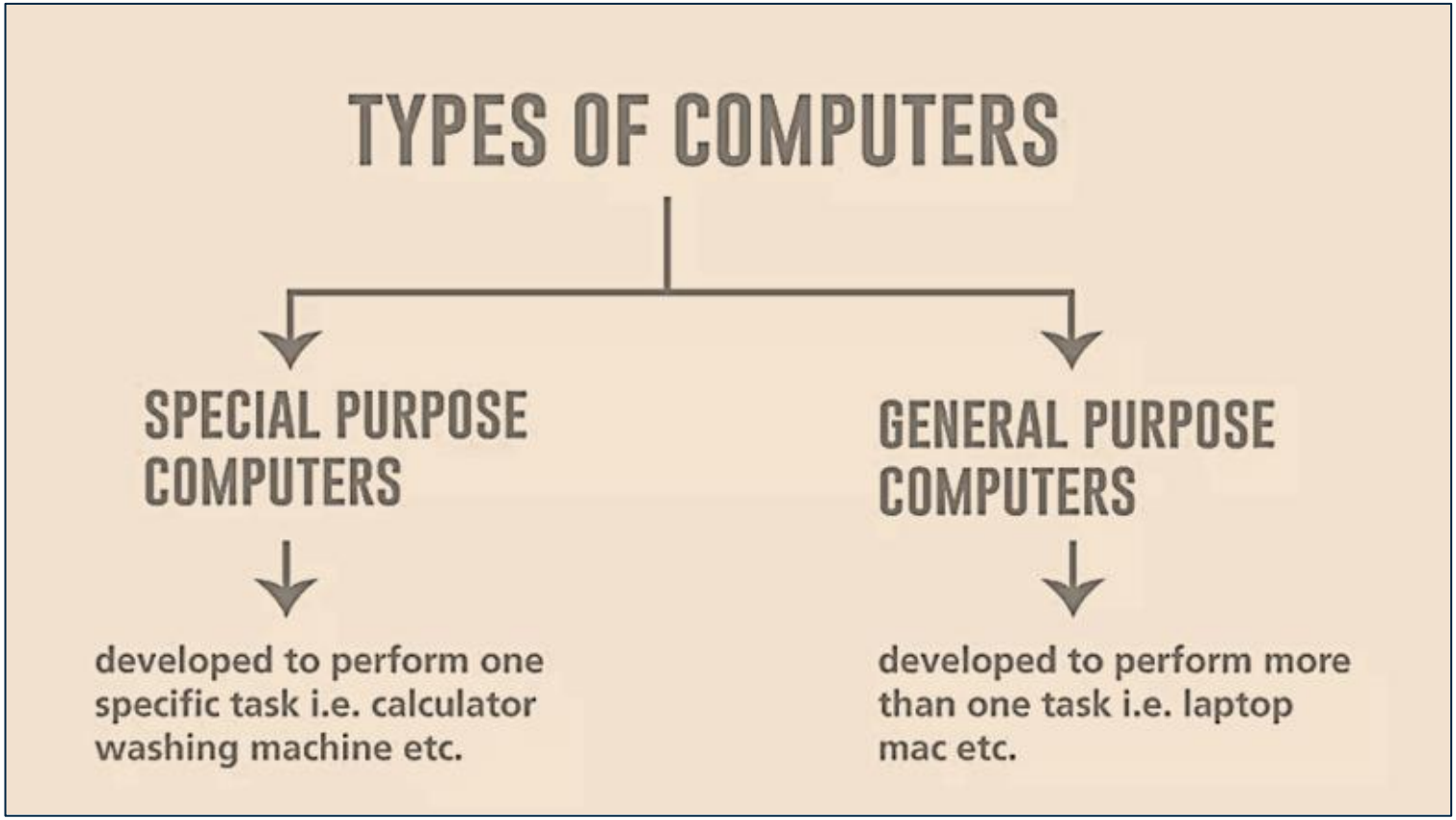


What is an Embedded System?

Slide Will be Revealed
During the Lecture



What is an Embedded System?



What is an Embedded System?

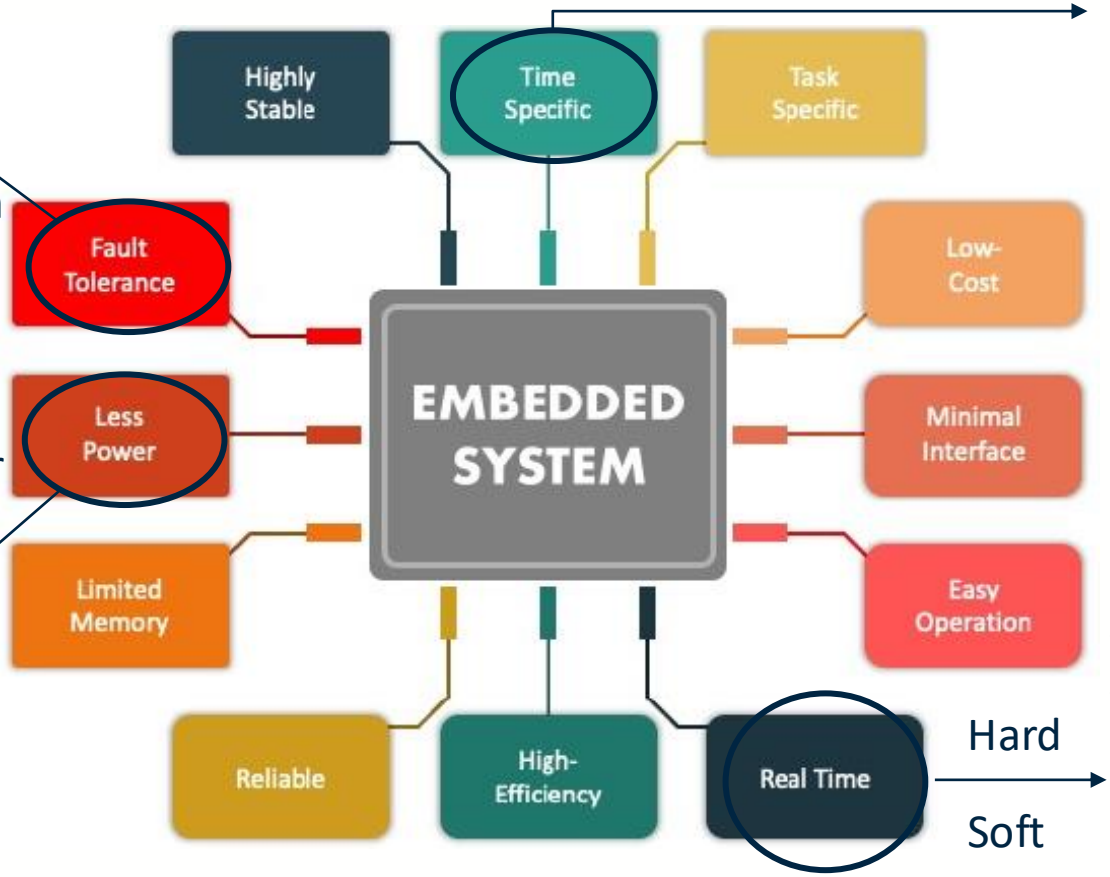
General-purpose systems vs Embedded Systems

Criteria	General Purpose Computer	Embedded system
Contents	It is combination of generic hardware and a general purpose OS for executing a variety of applications.	It is combination of special purpose hardware and embedded OS for executing specific set of applications
Operating System	It contains general purpose operating system	It may or may not contain operating system.
Alterations	Applications are alterable by the user.	Applications are non-alterable by the user.
Key factor	Performance" is key factor.	Application specific requirements are key factors.
Power Consumption	More	Less
Response Time	Not Critical	Critical for some applications

What is an Embedded System?

The ability of a system to continue functioning correctly even when faults or failures occur.

Embedded systems consume less power than PCs because they are designed for specific tasks and optimised for efficiency rather than general-purpose computing.



Perform tasks within a certain timeframe, but the timing is not necessarily critical.

Hard

Soft

Must meet strict timing constraints to function correctly. A missed deadline can lead to system failure.

Why Embedded Systems?

Market	Embedded Device
Automotive	Automatic Brake System (ABS)
Consumer electronics	TV, Toys, Kitchen appliances
Healthcare	Cardiac monitors, Prosthetic devices
Networking	Routers, Gateways
Offices	Monitor, Scanner, Projector
Industrial control	Robotics, Control systems

Enable

Embedded systems?

Types of Embedded System

- **Stand-alone Embedded Systems**

Stand-alone embedded systems **work by themselves** and do not require a host system to handle the communication of data between devices or other computer systems. Examples of stand-alone embedded systems are digital watches and video games consoles.

- **Real-time Embedded Systems**

A real-time embedded system is designed to **perform a time critical tasks**, such as streaming video and audio. These can be further classified into three types: hard, firm, and soft real-time systems.

- **Networked Embedded Systems**

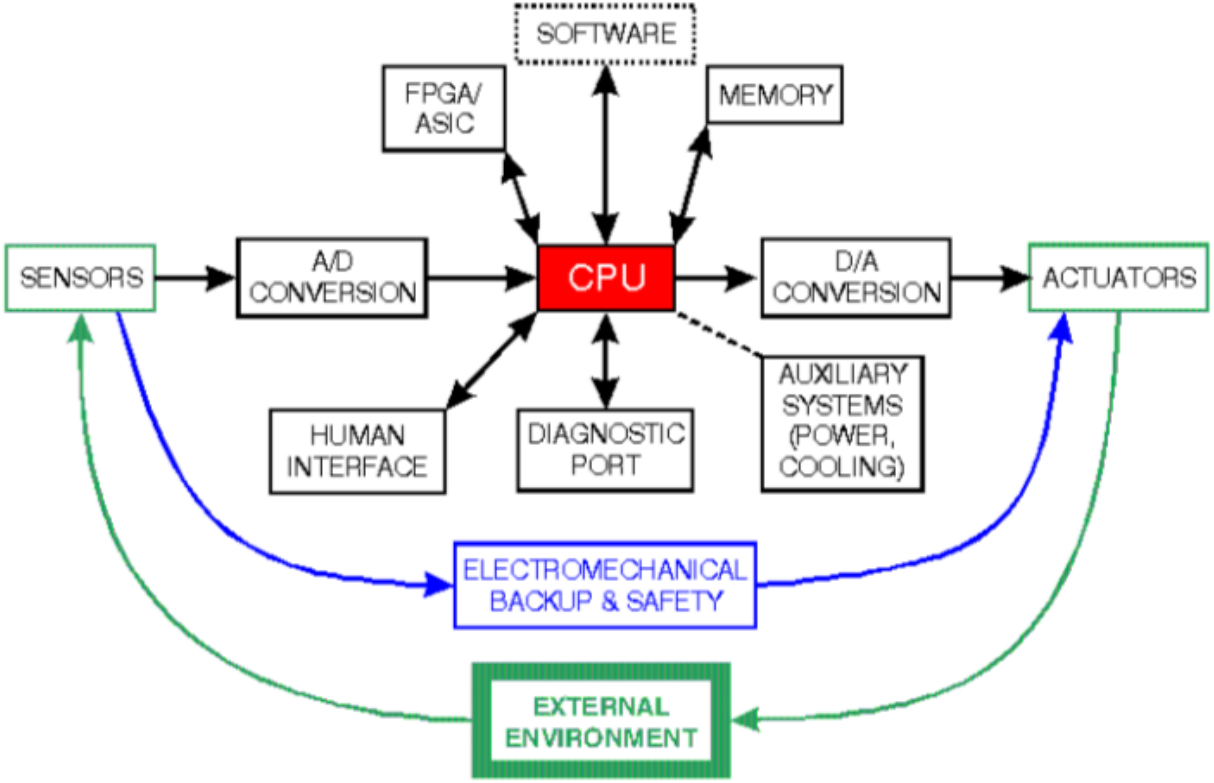
Networked embedded systems are **connected to a network of devices** so that they can share and access resources. An example of a networked embedded system is a card machine for taking payments by credit or debit card.

- **Mobile Embedded Systems**

Mobile embedded systems **are found in portable electronics**. These are easy to transport but may have limitations, such as the amount of memory or resources available. Examples of mobile embedded systems are fitness trackers and digital cameras.

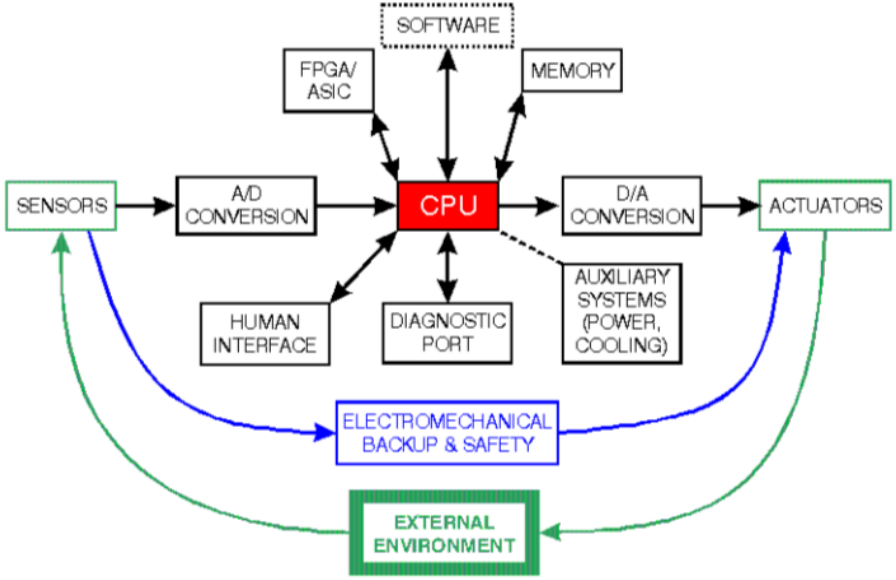
The categories are not mutually exclusive and a system could represent a combination of these.

Embedded System Example



Embedded System Example

- Key building blocks: CPU and Memory.
- Interfaces:
  - A/D, D/A, Serial.
  - Human Interface: (Simple) LED, LCD, (Complicated) robotic system.
  - FPGA/ASIC: To increase performance.

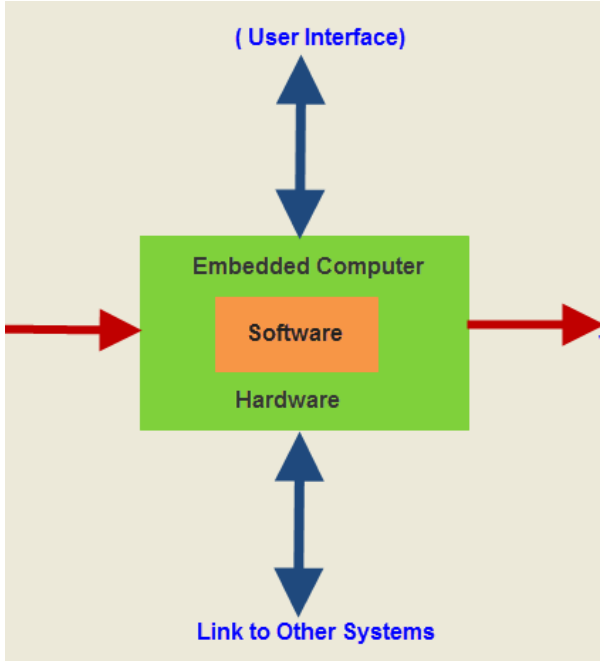


- Software: Usually a fixed function and is specific.

Embedded System Example

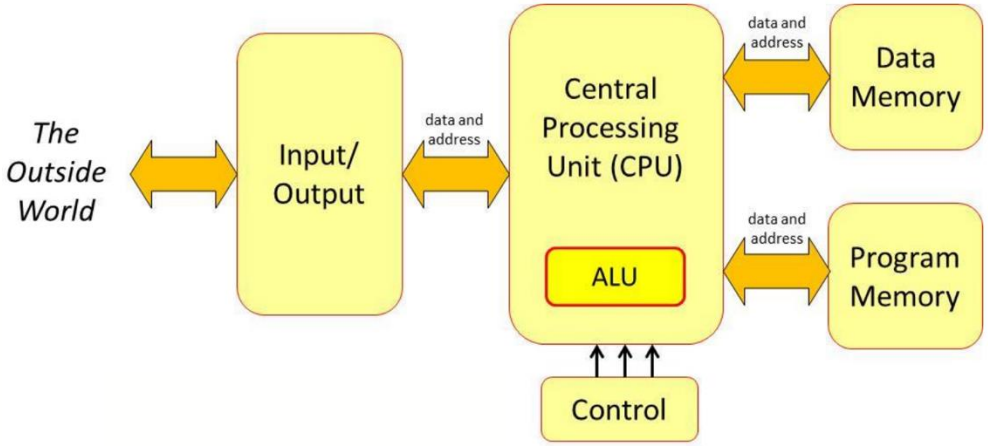
Hardware: Processor/CPU (microprocessors/microcontrollers), also include power supply, memory, timers, serial communication ports, sensor interface, analogue to digital converter, digital to analogue converter, actuator interface, and system application specific circuits.

Software: embedded operating systems (optional), different applications and device drivers.



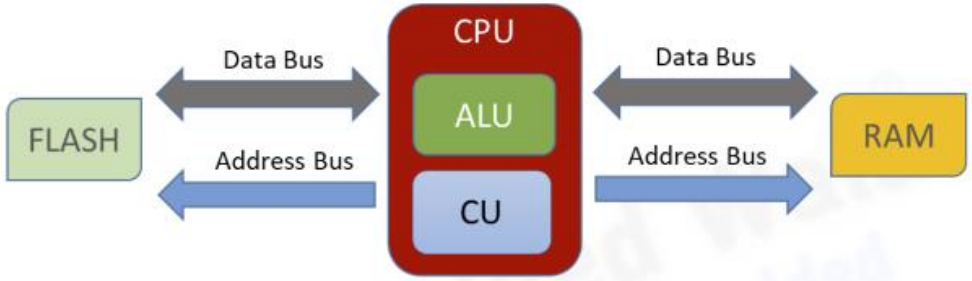
Refresher of Computer Architecture

- What are the basic components of a computer?
  - Processor/CPU.
  - Memory.
  - Input/Output Ports.

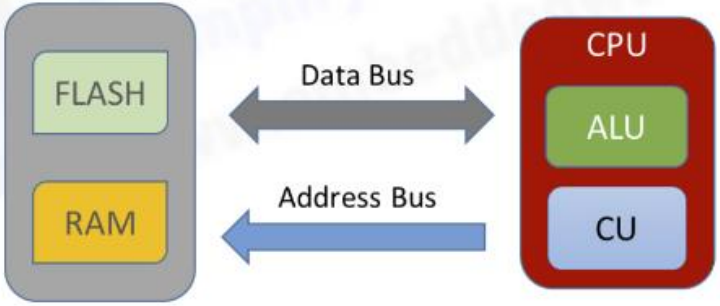


- Data bus: One set of wires carries the data itself.
- Address bus: The other set of wires carries address information.
- Address: is a digital number that indicates which place in memory the data should be stored in, or retrieved from.

Types of Computer Architectures



Harvard Architecture



Von-Neumann Architecture

Control Unit (CU)

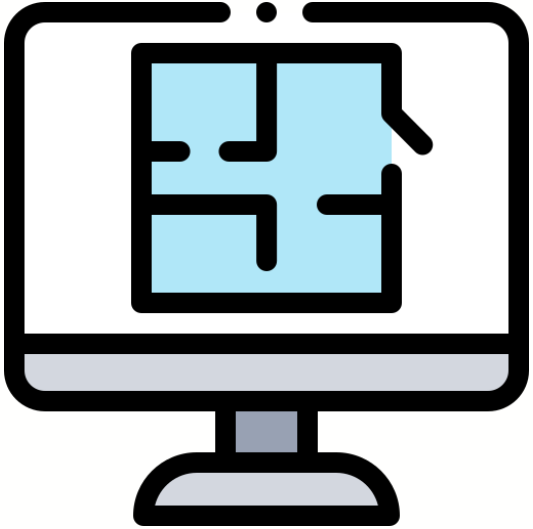
- *Control unit controls overall operations of computer. The CU fetches instructions from main memory (RAM), interprets them, fetches required data and tell ALU what to do.*

Arithmetic Logic Unit (ALU)

- *This unit handles the arithmetic and logic functions with in the CPU. It takes instructions decoded by the Control Unit, and performs actions as directed by control unit.*

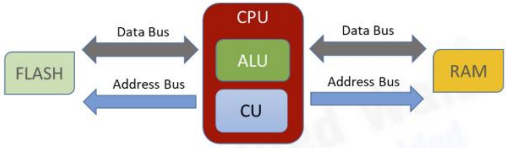
Types of Computer Architectures

Mentimeter

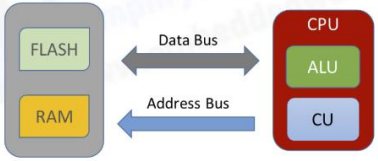


Difference between Von Neumann and Harvard

Slide Will be Revealed During the Lecture



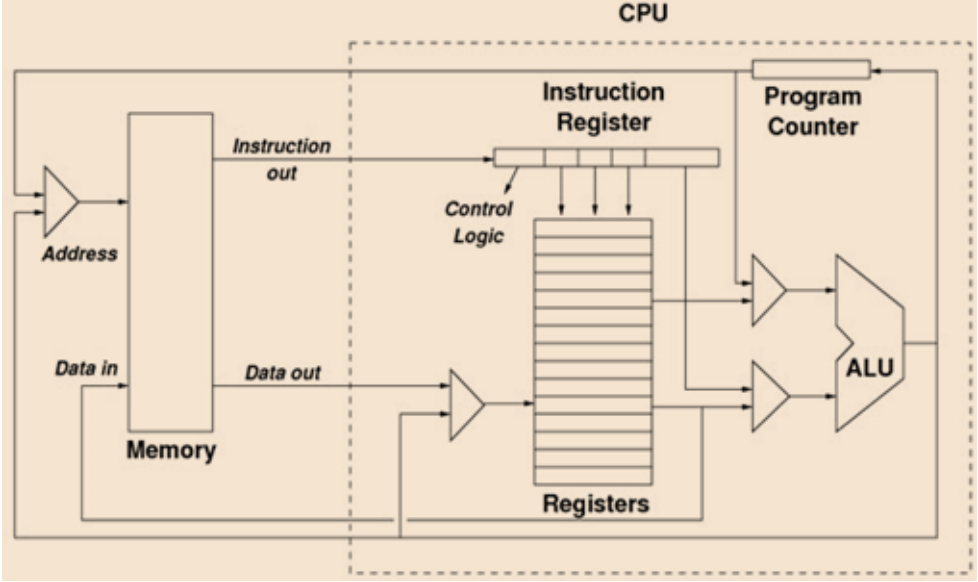
Harvard Architecture



Von-Neumann Architecture

Refresher of Computer Architecture

Central Processing Unit (CPU)



Arithmetic logic unit:

- Performs arithmetic operations such as simple addition, subtraction, multiplication, division.
- Perform logic operations such as OR and AND.

Registers:

- High speed storage located in the processor.
- Do not have address memory as you would in regular memory (they are addressed directly by their name).
- Have specific functions explicitly defined by the instruction.
- Can contain data or address registers.

Other Important terms

- **Port**

  A physical connection between the computer and the outside world

  - **Input Port**

  Hardware on the microcontroller that allows information about the external world to enter into the computer

  - **Output Port**

  Hardware to send information out to the external world

- **Device Driver**

  A set of software functions that facilitate the use of an I/O port (provides a software interface to hardware devices).

- **Interface**

  Hardware: The collection of the I/O port, external electronics, physical devices.
  Software: device driver.

- **Bus**

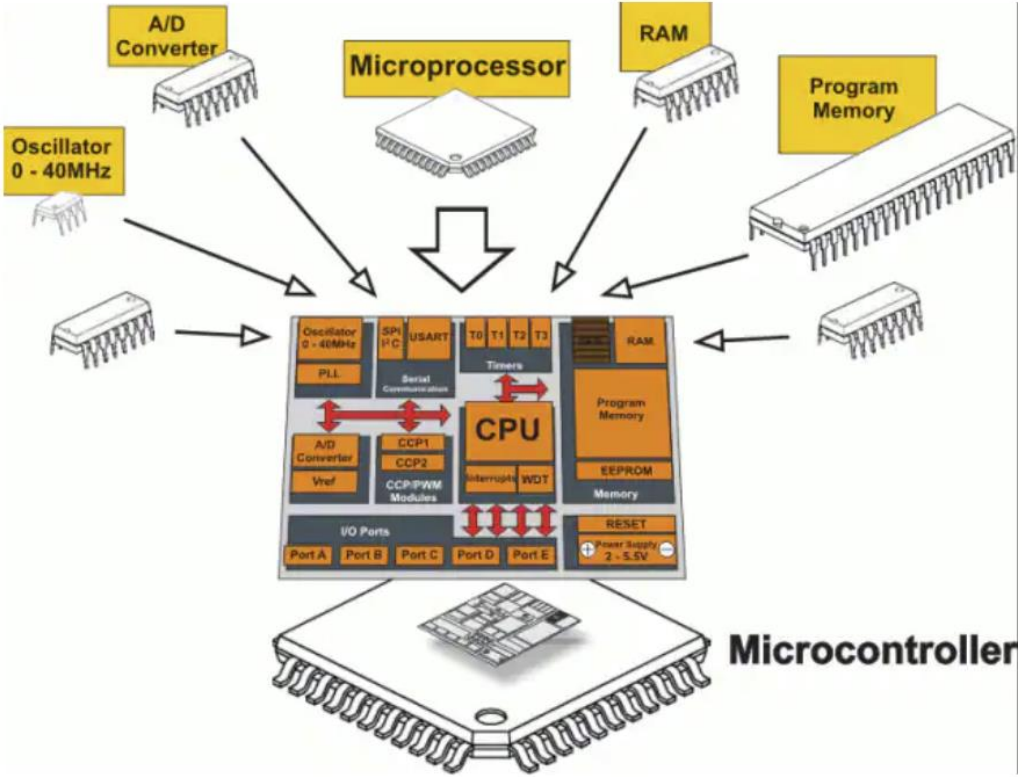
  A group of electrical conductors used to pass information between modules.

Microprocessor Unit (MPU) vs. Microcontroller Unit (MCU)

Microprocessors (MPU) vs. Microcontrollers (MCU)

Can anyone tell me the difference?

Microprocessor Unit (MPU) vs. Microcontroller Unit (MCU)



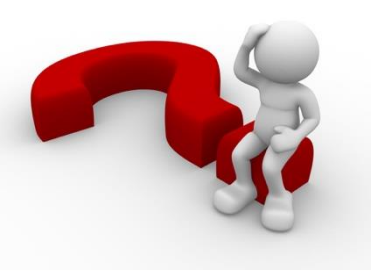
Microprocessor Unit (MPU) vs. Microcontroller Unit (MCU)

Summary

Parameters	Microprocessors	Microcontroller
Applications	Gaming, web browsing, document creation etc.	Dedicated for specific tasks (Camera, washing machine etc.)
Internal Structure	Memory and I/O devices connected externally	CPU, memory and I/O are present internally
Cost	High	Low
Power Consumption	High	Low
Memory (RAM)	512 MB to 32 GB	2KB to up to 256 KB
Storage	Hard Disk (128 GB to up to 2 TB)	Flash memory (32 KB to 2 MB)
Peripheral Interfaces	USB, UART, High speed Ethernet	UART, I2C, SPI

Microprocessor Unit (MPU) vs. Microcontroller Unit (MCU)

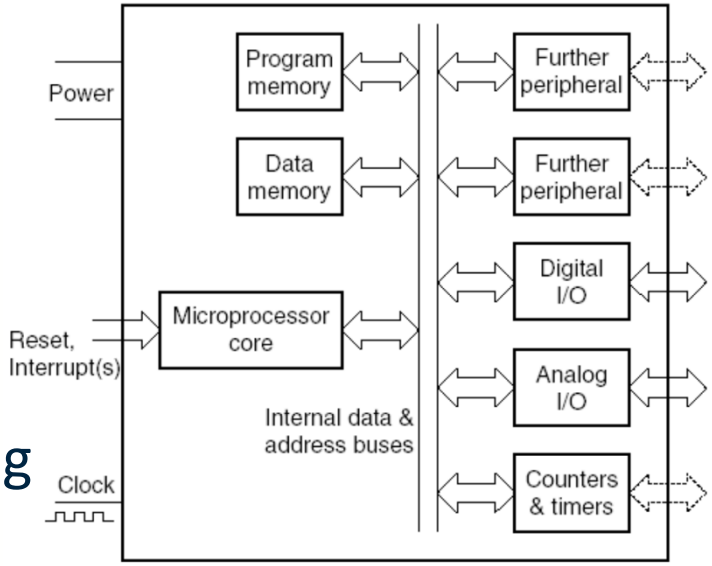
Question: In embedded systems which one (MPU or MCU) are preferred to apply?

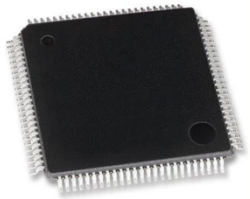


This Will be Revealed During the Lecture

Microcontroller Unit (MCU)

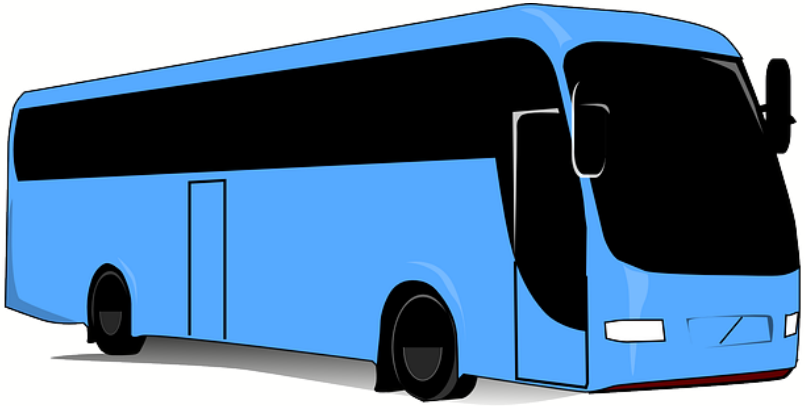
- A microcontroller takes the essential features of the computer, and adds to these the features that are needed for it to perform its control functions.
- It's useful to think of it as being made up of three parts: **core (MPU), memory and peripherals.**





Bus

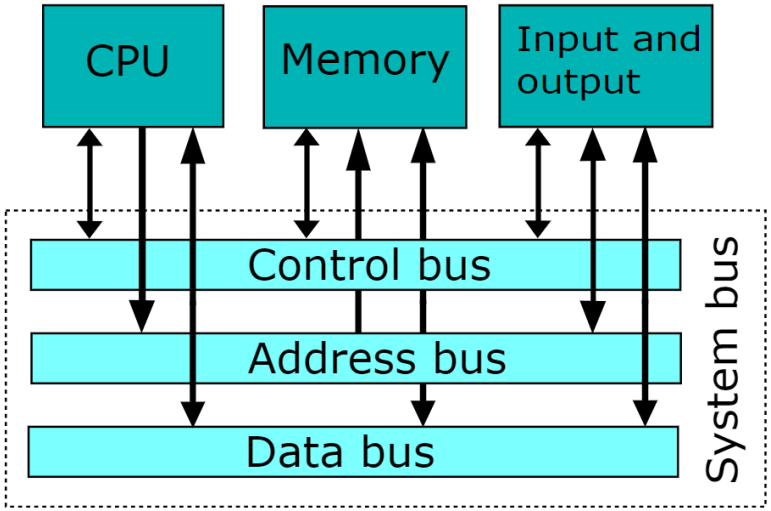
Concept: A **bus** is simply a collection of wires carrying various data signals, addresses, and control signals (clock signals, requests, acknowledgements, data type, etc.) between all of the other major components on the embedded board, which include the I/O sub-systems, memory subsystem, and the master processor.



Bus

Category:

- **Address Bus:** Carries memory addresses <u>from the processor to other components</u>.
- **Data Bus (bidirectional):** Carries the data.
- **Control Bus:** Carries the control signals.



There are three internal buses associated with processors: the **data bus, address bus, and control bus**. Together, these three make up the "system bus."

Bus

Parallel vs. Serial Communications

- Two communication schemes for Bus

  Parallel communications vs. Serial communications

- Parallel Communication

  When data is sent using parallel data transmission, multiple data bits are transmitted over multiple channels at the same time.

- Serial Communication

  Serial data transmission sends data bits one after another (in a specific order) over a single channel.

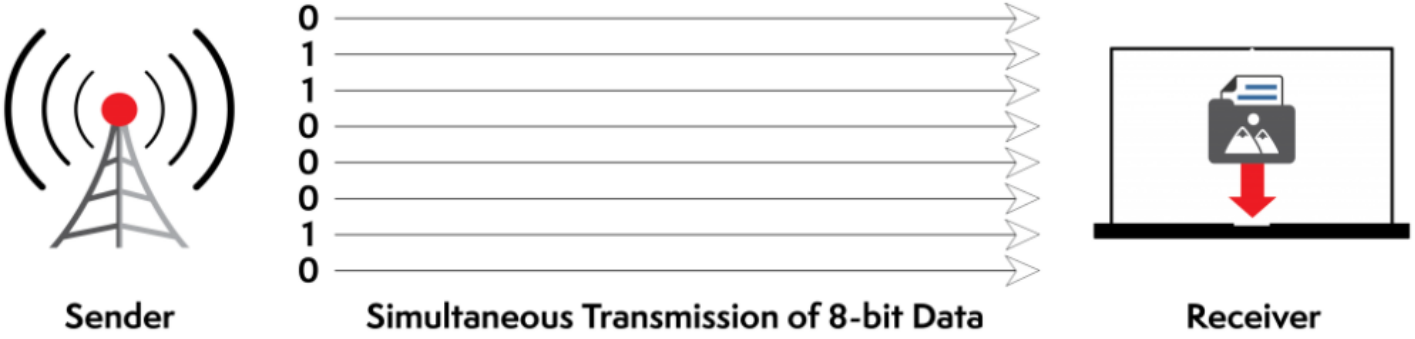
Bus

Parallel vs. Serial Communications (cont'd)

An example of transmitting number 98

Parallel transmission

98 ⟶ 01100010



A high voltage level to transmit 1 and a low voltage level 0

Bus

Parallel vs. Serial Communications (cont'd)

Serial transmission

98 → 01100010

A high voltage level to transmit 1 and a low voltage level 0

Bus

Parallel vs. Serial Communications (cont'd)

This Will be Revealed
During the Lecture

Bus

Parallel vs. Serial Communications (cont'd)

Q: Which one is suitable for BUS system in embedded systems?

This Will be Revealed
During the Lecture

Bus

Parallel vs. Serial Communications (cont'd)

Q: When is parallel transmission used to send data?

This Will be Revealed During the Lecture

University of Glasgow

University of Glasgow 2013 Glasgow College, UESTC

WORLD CHANGING GLASGOW

Summary

- Course Outline
- Overview of Embedded System
- Refresher of Computer Architecture
- Bus structure and tradeoffs

65

In the Next Lesson:

Introduction to ARM processor